PROJECT AND TEAM INFORMATION

Project Title

WAREHOUSE INVENTORY MANAGEMENT SYSTEM

Student / Team Information

Team Name: Team #	Tech Titans
Team member 1 (Team Lead) (Last Name, name: student ID: email, picture):	Mandola, Anushka – 2512510027 mandolaanushka@gmail.com 
Team member 2 (Last Name, name: student ID: email, picture):	Verma, Suyash – 25103140119 vsuyash142@gmail.com 

PROPOSAL DESCRIPTION (10 pts)

Motivation (1 pt)

A warehouse inventory never stops; stock comes in through sales and purchases, and leaves due to sales and orders. Such an inventory requires rigorous maintenance to avoid mismatch in counts, expiry dates, and orders. Manual tracking makes this worse. Numbers get written down wrong, expiry dates get missed, and by the time someone notices a product has gone bad, it's already a loss. Every product that passes its expiry date represents a loss, and lack of proper track of incoming and outgoing stock creates surplus stock of some items, and shortages of others, resulting in dissatisfaction among customers.

This project proposes a Warehouse Inventory Management System implemented in C++ that maintains a track of products, stock, expiry dates, and pending orders. This project highlights the importance of choosing the correct data structure for a given task and brings to light the use of common Data Structures and Algorithms in solving real-world problems. We're building a Warehouse Inventory Management System in C++ that keeps track of products, current stock, expiry dates, and pending orders, with a Linked List logging every stock movement as it happens, in order.

Really, this comes down to a simple idea: sloppy stock tracking costs money and annoys customers, especially with anything perishable. Pick the right data structure for the job - fast lookups, expiry priority, order handling . The main motivation of this project is to connect the concepts learned in our third semester DSA and C++ courses with a real -world problem, while gaining practical underdstanding of how data structures and algorithm can be combined to build an organized and efficient system.

State of the Art / Current solution (1 pt)

In current situation, most of the smaller and medium-sized warehouses maintain their inventory manually using papers or spreadsheets such as Microsoft Excel or Google Sheets. In these systems, an employee manually adds or removes stock, and maintains records by hand. Separate sheets or sections are used to track expiry dates and orders.

But in large scale businesses, they've management systems which provide excellent automation and tracking, they are often very expensive and require extensive training and time to set up, and are thus unsuitable for most small-scale businesses and often a complex overkill for small scale applications.

So you're stuck between two bad options: manual tracking that's error-prone or commercial software that's too much for what you actually need. There's a real gap here for something simple, low-cost, and easy to pick up without a manual. Over project overcomes this gap by combining Data Structures and Algorithm to make information easier to manage and process.

Project Goals and Milestones (2 pts)

Overall Aims:
The primary objective of this project is the development of a Warehouse Inventory Management System in C++. It is designed to help in the tracking of goods that are added or removed from the warehouse while also taking care of the stock count, expiry date, and order management. This inventory system will help resolve actual issues that warehouses face, such as wrong stock count, expiry date being overlooked, and delays in orders, by using the correct data structure.

Key Objectives:
1. Easy access to products as they are added or removed
2.Able to quickly retrieve products by their ID number
3.Monitor expiry dates and alert for those that are soon to expire
4.Order management system and order priority management
5.History tracking system for all products and their movements.

Milestones:
1.Week 1-2: Design the Product structure; get core add/remove/update working with a Hash Map.
2.Week 3: Add search by ID/name plus a full inventory display.
3.Week 4: Add an expiry field; build a Min-Heap to flag items nearing expiry.
4.Week 5: Build a Min-Heap priority system for pending orders.
5.Week 6: Add a Linked List to log stock-in/out events in order.
6.Week 6: Add a Linked List to log stock-in/out events in order.
7.Week 7: Test with sample data, fix bugs, handle edge cases (duplicate IDs, empty inventory).
8.Week 8: Final report, code cleanup, demo.

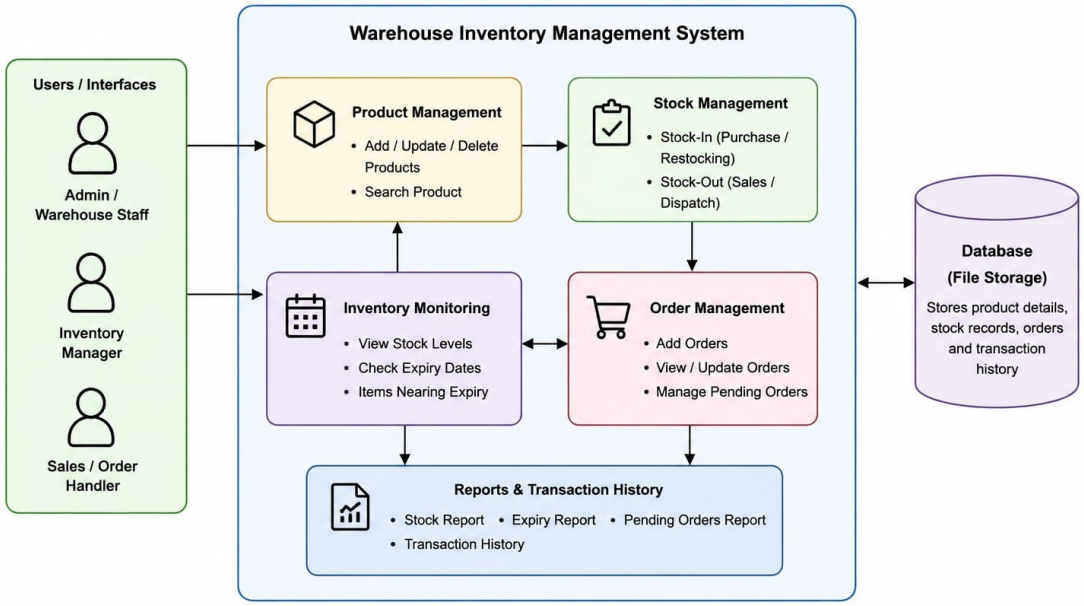
Project Approach (3 pts)

Solution Design Technologies: The solution to the problem will consist of a modular console application in C++, where the data structure used will be the Linked List because of the facility of inserting and removing items from the list dynamically, it lets products get tossed in or pulled out whenever, without ever needing to lock into a fixed size.

Design Approach: Each product basically turns into a node - carrying its ID, name, quantity, price, category, expiry date with a pointer tagging along to whatever next in line. There's an Inventory class handling the actual work adding a product just means slotting in a node, removing one means yanking it out, updating stock means tracking down the right node and nudging it, and searching by ID or name just means walking the list till you land on what you're after. A second linked list sits off to the side, logging every transaction in order as it happens, and something similar keeps tabs on pending orders, getting checked or updated whenever needed. To stop it turning into a tangled mess, it's kept to three simple layers - a data layer for the nodes and lists, a logic layer for the operations, and a menu-driven console on top - which just makes testing, debugging, and building on it later a lot less painful.

Platforms & Technologies: Language: C++ (C++17) Development Environment: VS Code / Code::Blocks with g++ compiler Version Control: Git and GitHub, to track progress across milestones Testing: Manual test cases using sample warehouse data, including edge cases like empty list, duplicate product IDs, and expired items Documentation: Markdown-based README explaining setup and usage Optional Future Extension: File I/O for saving/loading inventory data between runs, and a basic GUI if time permits.

System Architecture (High Level Diagram)(2 pts)



System Overview

The WIMS software program is a console-based C++ program that uses the Linked List data structure. This program is used by Admin/Warehouse Staff, Inventory Manager, and Sales/Order Handlers using a shared menu-driven program interface. **The system includes the following modules:**

- Product Management — Add, update, delete, and search products (stored as linked list.
- nodes with ID, Name, Quantity, Price).
- Stock Management — Handles Stock-In (restocking) and Stock-Out (sales/dispatch) by updating item quantities.
- Inventory Monitoring — Displays stock levels and generates low-stock alerts.
- Order Management — Tracks customer/purchase orders and their status (pending/completed).
- Reports — Generates stock reports and transaction history.
- File Storage — Saves data to a file so it persists even after the program closes.

Project Outcome / Deliverables (1 pts)

Outcomes / Deliverables: By the end of this project, the following deliverables will be produced:
1.Working C++ Program — A complete application for a Console based Warehouse Inventory Management System that uses a Linked List as the fundamental data structure for performing add, delete, update, and search operations on the product data.
2.Transaction Log — A linked list based feature to log all the transactions regarding products being either stocked in or stocked out in the chronological order.
3. Order Status Management — A linked list based application to record and update the order status.
4. Expiry Management Feature — A feature to enter products close to expiry.
5. Source Code Availability — Complete source code commented in multiple modules with the version history of the code base on GitHub showing each milestone completion.
6. Documentation of Project — The README file containing the system design and setup instructions along with the architecture diagram.
7. Sample Data — Sample warehouse data and test cases showing edge case handling capability of the system.
8. Report & Demonstration of Project — A final report along with the working demo of the project

Assumptions

Assumptions:

The system is a single-user, console-based application. All data is assumed to reside in memory during runtime; persistent storage (file-based saving/loading) is treated as optional. Product IDs are assumed to be unique and manually entered correctly by the user (no barcode /scanner integration). The warehouse deals with a manageable number of products, so linear traversal through the linked list is acceptable performance-wise. Expiry dates and order details are entered manually and assumed to be valid.

References

1. B. Stroustrup, The C++ Programming Language, 4th ed. Boston, MA, USA: Addison-Wesley, 2013.
2. cppreference.com, “C++ reference,” [Online]. Available: https://en.cppreference.com/w/cpp/. Cppreference
3. cplusplus.com, “C++ Language Tutorial,” [Online]. Available: C++ Language Tutorial. Cplusplus
4. GeeksforGeeks, “Data Structures,” [Online]. Available: Data Structures.